



TaskFlow

Gerenciador de Tarefas Pessoais

FASE 1

Integrante: Pedro

Henrique de

Almeida Peixoto

Professor:

Matheus Batista

Silva

Disciplina:

Dispositivos

Móveis e IOT

Entrega: 23 de

maio de 2026

Sumário

1.	Introdução
2.	Problema Identificado
3.	Objetivo Geral
4.	Objetivos Específicos
5.	Público-Alvo
6.	Descrição da Solução
7.	Tecnologias Utilizadas
8.	Requisitos Funcionais
9.	Requisitos Não Funcionais
10.	Usuários do Sistema
11.	Casos de Uso Principais
12.	Telas da Aplicação
13.	Organização do Código
14.	Componentes Reutilizáveis
15.	Dados Mockados
16.	Limitações da Fase 1
17.	Planejamento para a Fase 2
18.	Conclusão
19.	Critérios Avaliativos

1. Introdução

O avanço dos dispositivos móveis transformou profundamente a maneira como as pessoas gerenciam suas atividades cotidianas. Aplicativos de produtividade tornaram-se ferramentas indispensáveis para estudantes, profissionais e qualquer pessoa que necessite organizar sua rotina com eficiência e praticidade.

Nesse contexto, o TaskFlow surge como uma solução frontend desenvolvida em React Native com Expo, voltada ao gerenciamento de tarefas pessoais. O aplicativo permite que o usuário crie, visualize, filtre e atualize o andamento de suas tarefas de forma simples e intuitiva, com interface responsiva compatível com dispositivos móveis e navegadores web.

Este documento descreve o processo de desenvolvimento da Fase 1 do projeto TaskFlow, apresentando o problema identificado, os objetivos, os requisitos, os casos de uso, a estrutura técnica e o planejamento para fases futuras.

2. Problema Identificado

A desorganização pessoal é um desafio recorrente na vida de estudantes e profissionais. A ausência de um método claro para registrar, priorizar e acompanhar tarefas frequentemente resulta em esquecimentos, atrasos e baixa produtividade. Os principais problemas identificados são:

- Falta de organização pessoal para controlar atividades e compromissos.
- Esquecimento de tarefas importantes por ausência de registro adequado.
- Dificuldade para acompanhar prazos e estimar o tempo necessário para cada atividade.
- Falta de clareza sobre quais tarefas possuem maior urgência ou prioridade.
- Ausência de uma visão consolidada do que está pendente, em andamento ou já concluído.

O TaskFlow foi concebido para resolver esses problemas, oferecendo uma interface centralizada e de fácil utilização para o controle completo das tarefas pessoais do usuário.

3. Objetivo Geral

Desenvolver uma aplicação frontend responsiva utilizando React Native com Expo, capaz de simular o gerenciamento de tarefas pessoais por meio de criação, listagem, visualização, atualização de status e filtragem de tarefas, com interface adaptada para dispositivos móveis e navegadores web.

4. Objetivos Específicos

- Criar uma interface inicial de apresentação do sistema (tela de boas-vindas).
- Implementar telas de login e cadastro simulados para controle de acesso à aplicação.
- Permitir a criação de tarefas com título, descrição, prioridade, status e prazo de conclusão.
- Exibir uma lista de tarefas cadastradas em formato de cards visuais.
- Permitir a visualização detalhada de cada tarefa individualmente.
- Permitir a atualização do status de uma tarefa (Pendente, Em Andamento ou Concluída).
- Permitir a filtragem de tarefas por status ou nível de prioridade.
- Organizar o código-fonte em pastas por responsabilidade, seguindo boas práticas de desenvolvimento.
- Preparar a estrutura do frontend para integração futura com backend na Fase 2.

5. Público-Alvo

O TaskFlow é destinado a qualquer pessoa que necessite organizar suas atividades pessoais de forma prática. Os principais perfis de usuários incluem:

- Estudantes universitários que precisam controlar prazos de entregas, provas e atividades acadêmicas.
 - Profissionais que desejam organizar tarefas de trabalho e acompanhar o progresso de atividades diárias.
 - Pessoas que buscam uma ferramenta simples para gerenciar compromissos e rotinas pessoais.
 - Usuários que necessitam de uma visão clara e rápida do que está pendente, em execução ou concluído.
-

6. Descrição da Solução

O TaskFlow é um aplicativo de gerenciamento de tarefas pessoais desenvolvido em React Native com Expo, compatível com plataformas mobile e web. A solução permite ao usuário centralizar suas tarefas em um único ambiente, acompanhar o status de cada atividade, definir prioridades e obter uma visão geral do andamento das tarefas.

Na Fase 1, o sistema é composto exclusivamente pelo frontend da aplicação. Os dados são mockados (simulados em memória) e os fluxos de login e cadastro são implementados de forma simulada, sem validação real. Essa abordagem permite demonstrar o funcionamento completo da interface e validar todos os fluxos de uso antes da integração com o backend na Fase 2.

7. Tecnologias Utilizadas

Tecnologia	Finalidade
React Native	Framework principal para desenvolvimento da interface multiplataforma (mobile e web).
Expo	Ambiente para criação, execução e testes simplificados da aplicação React Native.
JavaScript / TypeScript	Linguagem de programação e tipagem estática utilizadas no desenvolvimento.
React Navigation	Biblioteca para navegação entre telas utilizando stack nativa.
React Hooks	Utilizados para controle de estado e comportamento (useState, useContext, useCallback).
Dados Mockados	Simulação de tarefas sem necessidade de banco de dados na Fase 1.
Replit	Ambiente de desenvolvimento, execução e hospedagem do projeto.

8. Requisitos Funcionais

Código	Descrição
RF01	O sistema deve possuir tela de boas-vindas com apresentação do aplicativo.
RF02	O sistema deve possuir tela de login simulado com validação de campos.
RF03	O sistema deve possuir tela de cadastro simulado com validação de campos.
RF04	O sistema deve permitir criar tarefas com título, descrição, prioridade, status e prazo.
RF05	O sistema deve permitir listar todas as tarefas cadastradas em formato de cards.
RF06	O sistema deve permitir visualizar os detalhes completos de uma tarefa selecionada.
RF07	O sistema deve permitir atualizar o status de uma tarefa (Pendente, Em Andamento, Concluída).
RF08	O sistema deve permitir filtrar tarefas por status ou nível de prioridade.
RF09	O sistema deve exibir um resumo com contagem de tarefas no dashboard principal.
RF10	O sistema deve permitir encerrar a sessão e retornar à tela inicial.

9. Requisitos Não Funcionais

Código	Descrição
RNF01	A aplicação deve ser desenvolvida em React Native com Expo.
RNF02	A aplicação deve ser responsiva e funcionar em dispositivos mobile e na web.
RNF03	O código deve ser organizado em pastas por responsabilidade.
RNF04	A aplicação deve utilizar componentes reutilizáveis para consistência visual e manutenibilidade.
RNF05	A aplicação deve funcionar completamente sem backend na Fase 1.
RNF06	Os dados exibidos devem ser mockados, simulando um ambiente real de uso.
RNF07	A interface deve ser clara, intuitiva e de fácil utilização pelo usuário final.
RNF08	O projeto deve estar estruturado para permitir integração futura com backend Node.js na Fase 2.

10. Usuários do Sistema

Usuário Comum

Pessoa que utiliza o TaskFlow para organizar suas tarefas pessoais. É o único perfil de usuário implementado na Fase 1.

Ações disponíveis ao usuário comum:

- Acessar a aplicação pela tela de boas-vindas.
- Realizar cadastro simulado com nome, e-mail e senha.
- Realizar login simulado com e-mail e senha.
- Criar novas tarefas com título, descrição, prioridade, status e prazo.
- Listar todas as tarefas disponíveis no dashboard.
- Visualizar detalhes de uma tarefa específica.
- Atualizar o status de uma tarefa.
- Filtrar tarefas por status ou prioridade.

Administrador

Na Fase 1, não há perfil administrativo implementado, pois o foco está no uso individual e pessoal do sistema. Caso necessário, esse perfil poderá ser considerado em versões futuras, contemplando funcionalidades como gerenciamento de múltiplos usuários e controle de acessos.

11. Casos de Uso Principais

Os cinco casos de uso a seguir representam as funcionalidades centrais do TaskFlow. Login e cadastro são requisitos obrigatórios de acesso simulado, mas não constituem casos de uso principais da aplicação.

Caso de Uso 1 — Criar Tarefa

Caso de Uso 2 — Listar Tarefas

Caso de Uso 3 — Visualizar Detalhes da Tarefa

Caso de Uso 4 — Atualizar Status da Tarefa

Caso de Uso 5 — Filtrar Tarefas por Status ou Prioridade

12. Telas da Aplicação

Tela	Descrição
Boas-vindas	Tela inicial com o nome TaskFlow, lista de funcionalidades e botões de acesso para login e cadastro. Apresenta a identidade visual do sistema.
Login	Tela de acesso simulado ao sistema. Solicita e-mail e senha, valida os campos e redireciona ao dashboard após confirmação.
Cadastro	Tela de registro simulado. Solicita nome, e-mail, senha e confirmação de senha, com validação de todos os campos obrigatórios.
Dashboard	Tela principal com saudação personalizada, cards de resumo por status, filtros por status e prioridade, e listagem completa de tarefas.
Criar Tarefa	Formulário para cadastro de nova tarefa. Permite informar título, descrição, prioridade, status, categoria e prazo de conclusão.
Detalhes da Tarefa	Tela com informações completas de uma tarefa. Permite alterar o status (Pendente, Em Andamento, Concluída) e excluir a tarefa.

13. Organização do Código

O projeto foi estruturado seguindo o princípio de separação por responsabilidade, com cada pasta dedicada a uma camada específica da aplicação:

Pasta	Conteúdo
app/screens	Telas principais da aplicação (Welcome, Login, Register, Dashboard, CreateTask, TaskDetails)
app/components	Componentes reutilizáveis de interface (Button, Input, Header, TaskCard, FilterTabs, EmptyState, StatusBadge)
app/navigation	Configuração do React Navigation, referências centralizadas e tipagem das rotas da aplicação
app/context	Contextos globais para gerenciamento de estado (TaskContext e AuthContext via React Context API)
app/data	Arquivo de dados mockados utilizado para simular tarefas cadastradas na Fase 1.
app/styles	Arquivo de tema com cores, espaçamentos, tipografia e sombras padronizados.
app/services	Pasta preparada para integrações futuras com AsyncStorage e API backend na Fase 2.

14. Componentes Reutilizáveis

A seguir estão os sete componentes reutilizáveis desenvolvidos para o TaskFlow:

Componente	
Button	Botão reutilizável com variantes (primary, secondary, danger, outline), estado de carregamento e ícone opcional.

Input	Campo de formulário com ícone lateral, toggle de visibilidade da senha, modo multiline e exibição de erro.
Header	Cabeçalho de telas internas com título, subtítulo opcional e botão de voltar configurável.
TaskCard	Card de exibição de tarefa com título, descrição resumida, indicador de prioridade, badge de status, categoria e prazo.
StatusBadge	Badge visual colorido para indicar status (Pendente, Em Andamento, Concluída) ou prioridade (Alta, Média, Baixa).
FilterTabs	Abas horizontais de filtro com scroll e suporte a ícone, para filtrar tarefas por status e prioridade.
EmptyState	Estado visual quando não há tarefas disponíveis, com ícone, mensagem explicativa e botão de ação opcional.

15. Dados Mockados

Na Fase 1, os dados são fornecidos por um arquivo de tarefas mockadas (`mockTasks.js`), que contém oito tarefas pré-cadastradas com informações realistas. Cada tarefa possui os campos: identificador único, título, descrição, status, prioridade, prazo de conclusão, data de criação e categoria.

O uso de dados mockados permite validar o funcionamento completo do frontend — incluindo listagem, filtros, detalhes e atualização de status — sem a necessidade de um backend real. As tarefas criadas pelo usuário durante a sessão são mantidas em memória por meio do `TaskContext` e descartadas ao encerrar a sessão, conforme esperado para a Fase 1.

16. Limitações da Fase 1

Por ser a fase inicial do projeto, o TaskFlow possui as seguintes limitações previstas e documentadas:

- Não há backend real: toda a lógica de dados é gerenciada localmente na memória da aplicação.
- Não há banco de dados conectado: os dados não são persistidos entre sessões.
- Login e cadastro são simulados: qualquer e-mail e senha válidos permitem acesso ao sistema.
- Os dados mockados e as tarefas criadas são perdidos ao encerrar a sessão.
- Autenticação real com verificação de credenciais será implementada na Fase 2.

17. Planejamento para a Fase 2

A Fase 2 do projeto contemplará a integração do frontend desenvolvido nesta fase com um backend completo:

- Backend em Node.js com Express para criação de uma API REST.
- Banco de dados MongoDB ou similar para persistência real de tarefas e usuários.
- Autenticação real com geração e validação de tokens JWT.
- Persistência das tarefas no banco de dados, mantendo os dados entre sessões.
- Integração do frontend com a API por meio de requisições HTTP.
- Possível uso de `AsyncStorage` para armazenamento local do token de autenticação.
- Melhorias na segurança, organização e escalabilidade da aplicação.

18. Conclusão

O TaskFlow, em sua Fase 1, cumpre com êxito o objetivo de apresentar um frontend funcional, responsivo e visualmente organizado para o gerenciamento de tarefas pessoais. A aplicação demonstra, por meio de dados mockados e fluxos simulados, todas as funcionalidades essenciais do sistema: criação, listagem, visualização, atualização de status e filtragem de tarefas.

O código foi desenvolvido com foco em organização, modularidade e reutilização de componentes, seguindo boas práticas de desenvolvimento em React Native. A estrutura do projeto foi pensada para facilitar a evolução natural para a Fase 2, onde serão integrados o backend, o banco de dados e a autenticação real.

Esta entrega representa a base sólida sobre a qual o sistema completo será construído, demonstrando domínio das tecnologias exigidas pela disciplina e capacidade de planejamento e execução de projetos de software para dispositivos móveis.

19. Critérios Avaliativos

O projeto TaskFlow foi desenvolvido considerando os critérios de avaliação da disciplina:

Qualidade da documentação

Documento acadêmico completo com todos os itens obrigatórios: capa, introdução, problema, objetivos, requisitos, casos de uso, telas, componentes e planejamento.

Nível de detalhes da aplicação

Seis telas implementadas, sete componentes reutilizáveis, oito tarefas mockadas e cinco casos de uso documentados e funcionais.

Organização do código

Estrutura modular em sete pastas por responsabilidade, com nomenclatura clara e separação de contextos, navegação e estilos.

Funcionamento sem erros

Aplicação verificada com zero erros de TypeScript, fluxos de navegação validados e todos os casos de uso funcionais.

Responsividade Web e Mobile

Interface adaptada para a web (maxWidth 720px) e mobile, com componentes responsivos em todas as telas.

Uso de React Native com Expo

Projeto desenvolvido integralmente em React Native com Expo, utilizando React Navigation, Hooks, Context API e dados mockados.

